

Below is an **experienced-developer interview preparation guide** for the 48 Infosys-style questions you listed. I'll explain not only the definition, but also **how to answer in an interview, practical examples, code, and follow-up points**.

A. ASP.NET Core / Web API

1. Explain the complete Web API lifecycle/request flow

When a client such as Angular calls an ASP.NET Core Web API, the request passes through several stages before the response returns.

Request flow

```
Angular / Browser
|
| HTTP Request
v
Kestrel Web Server
|
v
Middleware Pipeline
|
|-- Exception Handling
|-- HTTPS Redirection
|-- CORS
|-- Authentication
|-- Authorization
|
v
Routing
|
v
Controller / Endpoint
|
v
Model Binding + Validation
|
v
Service / Business Layer
|
v
Repository
|
v
EF Core / Dapper
|
v
SQL Server / PostgreSQL
|
v
```

```
Repository
  |
  v
Service
  |
  v
Controller
  |
  v
JSON Serialization
  |
  v
Middleware
  |
  v
HTTP Response
  |
  v
Angular
```

Example request:

http

```
GET /api/patients/1001
Authorization: Bearer <token>
```

Routing finds:

C#

```
[HttpGet("{id}")]
public async Task<IActionResult> GetPatient(Guid id)
{
    var patient = await _patientService.GetByIdAsync(id);

    if (patient == null)
        return NotFound();

    return Ok(patient);
}
```

The service calls the repository:

C#

```
public async Task<Patient?> GetByIdAsync(Guid id)
{
    return await _patientRepository.GetByIdAsync(id);
}
```

Repository:

C#

```
public async Task<Patient?> GetByIdAsync(Guid id)
{
    return await _context.Patients
        .AsNoTracking()
        .FirstOrDefaultAsync(x => x.Id == id);
}
```

Interview answer

When Angular sends an HTTP request, it first reaches Kestrel. The request then passes through the configured middleware pipeline. Middleware handles concerns such as exception handling, HTTPS, CORS, authentication and authorization. Routing identifies the appropriate controller/action or endpoint. Model binding converts route, query-string or body values into .NET objects and validation is performed. The controller delegates business logic to the service layer, which normally communicates with the repository and database through EF Core or another data-access mechanism. The result travels back through the pipeline, is serialized typically as JSON, and is returned to Angular.

For an experienced candidate, mention:

```
Controller → orchestration
Service → business rules
Repository → data access
DbContext → database communication
Middleware → cross-cutting request concerns
```

2. What is Middleware? Give a real project example.

Middleware is a component in the ASP.NET Core request pipeline that can:

- inspect a request
- modify a request
- call the next middleware
- modify the response
- terminate the request early

Basic example:

C#

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Request started");

    await next();

    Console.WriteLine("Response completed");
});
```

Real project example — Global Exception Middleware

Instead of writing try/catch in every controller:

C#

```
public class ExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionMiddleware> _logger;

    public ExceptionMiddleware(
        RequestDelegate next,
        ILogger<ExceptionMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception");

            context.Response.StatusCode = 500;
            context.Response.ContentType = "application/json";

            await context.Response.WriteAsJsonAsync(new
            {
                message = "An unexpected error occurred."
            });
        }
    }
}
```

Register:

C#

```
app.UseMiddleware<ExceptionMiddleware>();
```

Other real examples:

Exception handling
Request/response logging
Correlation IDs
Authentication
Authorization
CORS
Rate limiting
Tenant identification
Performance monitoring

Important interview point:

Middleware order matters.

For example:

C#

```
app.UseAuthentication();  
app.UseAuthorization();
```

Authentication must happen before authorization.

3. Explain Dependency Injection

Dependency Injection means a class receives its dependencies externally rather than creating them internally.

Bad approach:

C#

```
public class PatientService  
{  
    private readonly PatientRepository _repository;  
  
    public PatientService()  
    {  
        _repository = new PatientRepository();  
    }  
}
```

```
}  
}
```

Problems:

- tightly coupled
- difficult to unit test
- difficult to replace implementation

DI approach:

C#

```
public interface IPatientRepository  
{  
    Task<Patient?> GetByIdAsync(Guid id);  
}
```

Implementation:

C#

```
public class PatientRepository : IPatientRepository  
{  
}
```

Registration:

C#

```
builder.Services.AddScoped<  
    IPatientRepository,  
    PatientRepository>();
```

Injection:

C#

```
public class PatientService  
{  
    private readonly IPatientRepository _repository;  
  
    public PatientService(IPatientRepository repository)  
    {  
        _repository = repository;  
    }  
}
```

Benefits:

Loose coupling
Testability
Maintainability
Replaceable implementations
Centralized lifetime management
SOLID / Dependency Inversion

4. AddSingleton vs AddScoped vs AddTransient

This is an important interview question.

Lifetime	Instance
Singleton	One for application lifetime
Scoped	One per HTTP request
Transient	New every time requested

Singleton

C#

```
builder.Services.AddSingleton<ICacheService, CacheService>();
```

One object exists for the application lifetime.

Suitable for:

- configuration-like services
- application caches
- stateless thread-safe shared services

Be careful about thread safety.

Scoped

C#

```
builder.Services.AddScoped<IPatientService, PatientService>();
```

One instance per HTTP request.

Suppose:

```
Request 1
Controller
  ↓
PatientService

Request 2
Controller
  ↓
Different PatientService
```

EF Core `DbContext` is normally scoped.

Most business/repository services are commonly scoped.

Transient

C#

```
builder.Services.AddTransient<IEmailFormatter, EmailFormatter>();
```

Every resolution creates another instance.

Useful for:

- lightweight services
- stateless operations

Common interview trap

Can Singleton directly depend on Scoped?

Generally, **no**.

```
Singleton
  ↓
Scoped
```

creates a lifetime mismatch/captive dependency.

5. Explain Routing

Routing maps an incoming URL to an application endpoint.

Example:

C#

```
[ApiController]
[Route("api/[controller]")]
public class PatientsController : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult Get(Guid id)
    {
        ...
    }
}
```

Request:

http

GET /api/patients/123

maps to:

C#

Get(Guid id)

You can also use explicit routes:

C#

```
[HttpGet("active")]
public IActionResult GetActivePatients()
```

URL:

/api/patients/active

Route constraints:

C#

```
[HttpGet("{id:int}")]
```

6. Explain Use and Map

`Use()` adds middleware to the request pipeline.

C#

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Before");

    await next();

    Console.WriteLine("After");
});
```

`Map()` creates a branch based on request path.

C#

```
app.Map("/health", healthApp =>
{
    healthApp.Run(async context =>
    {
        await context.Response.WriteAsync("Healthy");
    });
});
```

Therefore:

Use

↓

Participates in normal middleware pipeline

Map

↓

Branches pipeline based on path

Also know `Run()` .

C#

```
app.Run(async context =>
{
    await context.Response.WriteAsync("End");
});
```

`Run()` terminates the pipeline.

7. Repository Pattern

Repository Pattern separates data-access logic from business logic.

```
Controller
  ↓
Service
  ↓
Repository Interface
  ↓
Repository Implementation
  ↓
EF Core
  ↓
Database
```

Interface:

C#

```
public interface IPatientRepository
{
    Task<Patient?> GetByIdAsync(Guid id);
    Task AddAsync(Patient patient);
}
```

Implementation:

C#

```
public class PatientRepository : IPatientRepository
{
    private readonly AppDbContext _context;

    public PatientRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<Patient?> GetByIdAsync(Guid id)
    {
        return await _context.Patients.FindAsync(id);
    }

    public async Task AddAsync(Patient patient)
    {
        await _context.Patients.AddAsync(patient);
        await _context.SaveChangesAsync();
    }
}
```

Experienced-candidate answer:

I use repositories primarily to isolate persistence concerns from business logic. Controllers communicate with services, services implement business rules, and repositories handle database queries. This improves separation of concerns and testability. I also avoid creating unnecessary generic repository abstractions where EF Core's DbContext already provides the required abstraction.

That last sentence demonstrates senior-level understanding.

8. MVC/Web API Action Verbs

Main HTTP verbs:

Verb	Purpose
GET	Read
POST	Create
PUT	Full update
PATCH	Partial update
DELETE	Delete

Examples:

C#

```
[HttpGet]
public IActionResult GetAll()

[HttpGet("{id}")]
public IActionResult Get(Guid id)

[HttpPost]
public IActionResult Create(CreatePatientRequest request)

[HttpPut("{id}")]
public IActionResult Update(Guid id, UpdatePatientRequest request)

[HttpDelete("{id}")]
public IActionResult Delete(Guid id)
```

REST example:

```
GET    /api/patients
GET    /api/patients/10
POST   /api/patients
PUT    /api/patients/10
DELETE /api/patients/10
```

9. HTTP Status Codes

Common ones you should know:

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthenticated
403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Content
429	Too Many Requests
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

Example:

```
C#

if (patient == null)
    return NotFound();
```

```
return Ok(patient);
```

POST:

C#

```
return CreatedAtAction(  
    nameof(Get),  
    new { id = patient.Id },  
    patient);
```

Important:

401 → Who are you? Authentication failed/missing.

403 → I know who you are, but you're not permitted.

10. What are Filters?

Filters execute at particular stages around MVC controller/action processing.

Important filter types:

- Authorization filters
- Resource filters
- Action filters
- Exception filters
- Result filters

Example:

C#

```
public class LoggingFilter : IActionFilter  
{  
    public void OnActionExecuting(ActionExecutingContext context)  
    {  
        Console.WriteLine("Before action");  
    }  
  
    public void OnActionExecuted(ActionExecutedContext context)  
    {  
        Console.WriteLine("After action");  
    }  
}
```

Difference from middleware:

Middleware



Entire ASP.NET Core HTTP pipeline

Filters



MVC/action execution pipeline

Use middleware for global HTTP concerns and filters when logic specifically needs MVC/action context.

11. How does your .NET application connect to DB?

A common production flow is:

Controller



Service



Repository



DbContext



ADO.NET Provider



Database

Configuration:

JSON

```
{
  "ConnectionStrings": {
    "DefaultConnection": "...
  }
}
```

Registration:

C#

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString(
            "DefaultConnection")));
```

DbContext:

C#

```
public class AppDbContext : DbContext
{
    public DbSet<Patient> Patients => Set<Patient>();
}
```

Mention that production secrets should not be hard-coded in source control; use appropriate secret/configuration management.

12. State Management in MVC

HTTP is stateless.

Different requests do not automatically remember previous requests.

Common state-management approaches:

```
Cookies
Session
TempData
ViewData
ViewBag
Query string
Hidden fields
Cache
Database
```

In modern Web API + Angular systems, backend APIs should generally remain stateless, with persistent business state stored in databases or other appropriate stores.

13. Session Management

Session stores user-specific temporary data across multiple HTTP requests.

Configuration:

C#

```
builder.Services.AddDistributedMemoryCache();

builder.Services.AddSession();
```

Pipeline:

C#

```
app.UseSession();
```

Usage:

C#

```
HttpContext.Session.SetString(  
    "UserName",  
    "Venkat");
```

Read:

C#

```
var name =  
    HttpContext.Session.GetString("UserName");
```

For distributed applications, consider distributed session storage such as Redis rather than relying only on a single server's memory.

14. TempData vs ViewBag

Feature	ViewBag	TempData
Current request	Yes	Yes
Survives redirect	No	Yes
Typical usage	Controller → View	Redirect messages

Example:

C#

```
ViewBag.Name = "Venkat";
```

TempData:

C#

```
TempData["Message"] =  
    "Patient created successfully";
```

Then:

C#

```
return RedirectToAction("Index");
```

TempData survives into the subsequent request, which is why it's often used for post-redirect messages.

15. Caching

Caching avoids repeatedly retrieving or calculating data that changes infrequently.

Without caching:

```
Request
  ↓
API
  ↓
Database
```

With caching:

```
Request
  ↓
API
  ↓
Cache?
  ↙  ↘
Yes  No
  ↓   ↓
Return Database
      ↓
      Cache
```

Types:

- In-memory cache
- Distributed cache
- Response/output caching
- Client/browser caching
- CDN caching

Example:

C#

```
if (!_cache.TryGetValue("Departments", out List<Department>? departments))
{
    departments = await _repository.GetDepartmentsAsync();

    _cache.Set(
        "Departments",
        departments,
        TimeSpan.FromMinutes(10));
}
```

For multi-instance applications, distributed caches such as Redis are usually preferable for shared cached data.

B. C# / LINQ

16. Polymorphism

Polymorphism means one abstraction can represent multiple implementations.

C#

```
public abstract class PaymentService
{
    public abstract void Pay();
}
```

Implementations:

C#

```
public class CreditCardPayment : PaymentService
{
    public override void Pay()
    {
        Console.WriteLine("Credit card payment");
    }
}

public class UpiPayment : PaymentService
{
    public override void Pay()
    {
        Console.WriteLine("UPI payment");
    }
}
```

Usage:

C#

```
PaymentService payment =  
    new UpiPayment();  
  
payment.Pay();
```

Runtime determines which implementation executes.

Also understand:

Compile-time polymorphism → method overloading

Runtime polymorphism → method overriding

17. FirstOrDefault()

`FirstOrDefault()` returns the first matching element or the type's default value when no match exists.

C#

```
var employee =  
    employees.FirstOrDefault(x => x.Id == 10);
```

For reference types, no match usually means:

C#

```
null
```

Difference:

`First()`

throws if there is no result.

`FirstOrDefault()`

returns default.

Don't confuse this with `SingleOrDefault()` :

FirstOrDefault → zero/many allowed, returns first

SingleOrDefault → expects zero or exactly one; throws if multiple

18. LINQ

LINQ means Language Integrated Query.

Example:

C#

```
var activeEmployees = employees
    .Where(x => x.IsActive)
    .OrderBy(x => x.Name)
    .Select(x => new
    {
        x.Id,
        x.Name
    })
    .ToList();
```

Common operations:

Where
Select
OrderBy
GroupBy
Join
Any
All
Count
FirstOrDefault
SingleOrDefault
Distinct
Skip
Take

EF Core can translate suitable LINQ expressions into SQL.

C#

```
var employees = await _context.Employees
    .Where(x => x.IsActive)
    .ToListAsync();
```

19. IEnumerable vs IQueryable

Very important for experienced candidates.

IEnumerable

Primarily operates over in-memory enumeration.

C#

```
IEnumerable<Employee> employees =  
    _context.Employees.ToList();
```

Then:

C#

```
employees.Where(x => x.DepartmentId == 10);
```

The table has already been retrieved before that in-memory filtering.

IQueryable

Builds an expression tree that providers such as EF Core can translate.

C#

```
IQueryable<Employee> query =  
    _context.Employees;  
  
query = query.Where(  
    x => x.DepartmentId == 10);  
  
var employees =  
    await query.ToListAsync();
```

Conceptually SQL becomes:

SQL

```
SELECT ...  
FROM Employees  
WHERE DepartmentId = 10;
```

Therefore:

IEnumerable → enumeration/in-memory operations

IQueryable → provider-backed query composition

For large DB queries, filtering/pagination should generally happen before materialization.

C. Angular

20. Types of Angular Data Binding

Four commonly discussed forms:

Interpolation

HTML

```
<h1>{{ title }}</h1>
```

Component → View.

Property binding

HTML

```
<img [src]="imageUrl">
```

Component → View.

Event binding

HTML

```
<button (click)="save()" ">  
  Save  
</button>
```

View → Component.

Two-way binding

HTML

```
<input [(ngModel)]="patientName">
```

Both directions.

Easy interview summary:

{{ }}	Interpolation
[property]	Property binding

(event)	Event binding
[()]	Two-way binding

Remember:

Banana in a box: [()]

21. Why Angular is called SPA?

SPA means Single Page Application.

Initially:

```
Browser
  ↓
index.html
  ↓
Angular application
```

After that Angular Router changes the displayed components without requiring a traditional full-page reload for each route.

```
/patients
/doctors
/appointments
```

Angular updates the relevant DOM/component tree while backend data is retrieved asynchronously.

22. Update grid without page reload

Call the API asynchronously and update the grid's data source/state.

Example:

```
TypeScript

loadUsers() {
  this.userService.getUsers().subscribe({
    next: data => {
      this.users.set(data);
    }
  });
}
```

Template reacts:

HTML

```
@for (user of users(); track user.id) {  
  <tr>  
    <td>{{ user.name }}</td>  
  </tr>  
}
```

Flow:

```
Button/filter/page event  
↓  
HttpClient  
↓  
Web API  
↓  
JSON  
↓  
Signal / Observable  
↓  
Angular rendering  
↓  
Grid updated
```

No browser refresh is necessary.

23. Angular Lifecycle Hooks

Common hooks:

```
constructor  
↓  
ngOnChanges  
↓  
ngOnInit  
↓  
ngDoCheck  
↓  
ngAfterContentInit  
↓  
ngAfterContentChecked  
↓  
ngAfterViewInit  
↓  
ngAfterViewChecked  
↓
```

Most frequently discussed:

ngOnInit

Initialization.

TypeScript

```
ngOnInit() {  
    this.loadPatients();  
}
```

ngOnChanges

Responds to changes in input values.

ngAfterViewInit

Used when view/child components have been initialized.

ngOnDestroy

Cleanup.

TypeScript

```
ngOnDestroy() {  
    // cleanup subscriptions/resources if needed  
}
```

Modern Angular also offers APIs such as `DestroyRef` and `takeUntilDestroyed()` that can simplify subscription cleanup.

24. Angular API has no response/error. How troubleshoot?

Answer this systematically.

1. Browser DevTools
↓
2. Network tab
↓
3. Was request generated?
↓
4. URL/method/payload correct?
↓

```
5. Status code?
    ↓
6. Browser console/CORS?
    ↓
7. Authorization/JWT?
    ↓
8. API logs
    ↓
9. Controller
    ↓
10. Service
    ↓
11. Repository
    ↓
12. Database/query
```

Examples:

```
400 → payload/model validation

401 → authentication/token

403 → authorization

404 → endpoint/routing

500 → backend exception

502/503 → proxy/server/service availability

CORS error → origin/header/method policy
```

Also reproduce the API independently using Swagger or another API client. This helps determine whether the issue belongs to Angular or the backend.

25. Pipes

Pipes transform values for display.

Example:

```
HTML

{{ patientName | uppercase }}
```

Date:

HTML

```
{{ appointmentDate | date:'dd/MM/yyyy' }}
```

Currency:

HTML

```
{{ amount | currency:'INR' }}
```

Pipes are primarily a presentation transformation mechanism.

26. Custom Pipe

Example status/color value pipe:

TypeScript

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'statusColor',
  standalone: true
})
export class StatusColorPipe
  implements PipeTransform {

  transform(status: string): string {
    switch (status) {
      case 'Approved':
        return 'green';

      case 'Rejected':
        return 'red';

      case 'Pending':
        return 'orange';

      default:
        return 'black';
    }
  }
}
```

HTML:

HTML

```
<span [style.color]="status | statusColor">
  {{ status }}
</span>
```

In larger applications, CSS classes are often preferable to returning raw styling values.

27. Angular Signals

Signals are Angular's reactive state primitive.

TypeScript

```
count = signal(0);
```

Read:

TypeScript

```
console.log(this.count());
```

Set:

TypeScript

```
this.count.set(10);
```

Update:

TypeScript

```
this.count.update(x => x + 1);
```

Computed:

TypeScript

```
doubleCount =
  computed(() => this.count() * 2);
```

Signals make dependencies explicit and provide fine-grained reactive state updates.

Example:

TypeScript

```
users = signal<User[]>([]);
```

After API call:

TypeScript

```
this.users.set(response);
```

28. Standalone Components

Modern Angular allows components without requiring declaration in an NgModule.

TypeScript

```
@Component({
  selector: 'app-patient',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './patient.html'
})
export class PatientComponent {
}
```

Benefits include:

Less NgModule boilerplate
Simpler dependency imports
Easy lazy loading
Clear component dependencies
Modern Angular architecture

29. Angular Dependency Injection

Angular also provides a DI system.

Service:

TypeScript

```
@Injectable({
  providedIn: 'root'
})
export class PatientService {
}
```

Traditional constructor injection:

TypeScript

```
constructor(  
  private patientService: PatientService  
) {}
```

Modern approach:

TypeScript

```
private patientService =  
  inject(PatientService);
```

`providedIn: 'root'` typically creates an application-level singleton service.

30. Dependencies/injection with Angular modules

Traditional module architecture:

TypeScript

```
@NgModule({  
  providers: [  
    PatientService  
  ]  
})  
  
export class PatientModule {  
}
```

Angular creates providers using hierarchical injectors.

Modern standalone applications can configure application providers during bootstrap:

TypeScript

```
bootstrapApplication(AppComponent, {  
  providers: [  
    provideHttpClient()  
  ]  
});
```

Experienced answer:

Earlier Angular applications commonly configured many dependencies through NgModules. Modern standalone Angular applications can register providers directly

during bootstrap or at component/route level, while the underlying hierarchical DI concept remains.

31. providers vs viewProviders

`providers` makes a dependency available to the component and relevant descendants, including projected content according to Angular's injector rules.

TypeScript

```
@Component({  
  providers: [PatientService]  
})
```

`viewProviders` limits it to the component's view and its view children; projected content does not receive it from that view provider.

TypeScript

```
@Component({  
  viewProviders: [PatientService]  
})
```

Interview summary:

`providers`

→ component + broader child/content injection visibility

`viewProviders`

→ component view + view children

→ excludes projected content

32. Two-way binding

Two-way binding synchronizes component state and the UI.

TypeScript

```
patientName = '';
```

HTML:

HTML

```
<input [(ngModel)]="patientName">
```

Equivalent conceptually to:

HTML

```
<input
  [(ngModel)]="patientName"
  (ngModelChange)="patientName = $event">
```

So:

Component → UI
UI → Component

33. How Angular connects to backend API

Using `HttpClient`.

Configuration:

TypeScript

```
provideHttpClient()
```

Service:

TypeScript

```
@Injectable({
  providedIn: 'root'
})
export class PatientService {

  private http = inject(HttpClient);

  getPatients() {
    return this.http.get<Patient[]>(
      '/api/patients'
    );
  }
}
```

Component:

TypeScript

```
this.patientService
  .getPatients()
  .subscribe({
    next: result =>
      this.patients.set(result),
    error: error =>
      console.error(error)
  });
```

Typical architecture:

```
Angular Component
↓
Angular Service
↓
HttpClient
↓
Interceptor
↓
HTTPS
↓
ASP.NET Core API
↓
Service
↓
Repository
↓
Database
```

Interceptors can handle concerns such as authentication headers and centralized error handling.

34. What did you use before Signals?

A strong answer:

Before Signals, I primarily used RxJS Observables and BehaviorSubject for reactive application state. For simpler component state I used normal component properties. AsyncPipe was used to subscribe in templates. For shared state across components, a service containing BehaviorSubject was a common approach.

Example:

TypeScript

```
private usersSubject =
    new BehaviorSubject<User[]>([]);

users$ =
    this.usersSubject.asObservable();
```

Update:

TypeScript

```
this.usersSubject.next(users);
```

Template:

HTML

```
@for (user of users$ | async; track user.id) {
    ...
}
```

Don't tell the interviewer Signals completely replace RxJS.

RxJS remains extremely useful for asynchronous streams and event composition.

35. Angular/JavaScript tricky output questions

Example:

JavaScript

```
console.log(1);

setTimeout(() => console.log(2), 0);

Promise.resolve()
    .then(() => console.log(3));

console.log(4);
```

Output:

```
1
4
3
2
```

Because:

Synchronous code

↓

Promise microtasks

↓

Timer/macrotasks

Another common question:

JavaScript

```
console.log([1,2,3].map(parseInt));
```

Result:

```
[1, NaN, NaN]
```

Because `map` passes:

```
value, index, array
```

while `parseInt` interprets the second argument as radix:

JavaScript

```
parseInt("1", 0)
parseInt("2", 1)
parseInt("3", 2)
```

Correct:

JavaScript

```
[1,2,3].map(x => parseInt(x, 10));
```

D. SQL

36. Indexes

An index is a database structure designed to improve data retrieval.

Without an appropriate index, SQL Server may perform:

```
Table Scan
```

With a useful index:

Index Seek

Example:

SQL

```
CREATE INDEX IX_Employee_DepartmentId
ON Employee (DepartmentId);
```

Query:

SQL

```
SELECT *
FROM Employee
WHERE DepartmentId = 10;
```

Clustered index

Determines the physical/logical organization of table data at the leaf level; a table can have only one clustered index.

Nonclustered index

Separate structure containing key values and row locators; a table can have many.

Important senior answer:

Indexes improve reads but are not free. They consume storage and add overhead to INSERT, UPDATE and DELETE because index structures also need to be maintained.

37. CTE

CTE = Common Table Expression.

SQL

```
WITH ActiveEmployees AS
(
    SELECT Id,
           Name,
           DepartmentId
    FROM Employee
    WHERE IsActive = 1
```

```
)  
SELECT *  
FROM ActiveEmployees;
```

Useful for:

- readability
- breaking complex queries into logical steps
- recursive queries
- hierarchical structures

Recursive example:

SQL

```
WITH EmployeeHierarchy AS  
(  
    SELECT Id, Name, ManagerId  
    FROM Employee  
    WHERE ManagerId IS NULL  
  
    UNION ALL  
  
    SELECT e.Id,  
           e.Name,  
           e.ManagerId  
    FROM Employee e  
    INNER JOIN EmployeeHierarchy h  
        ON e.ManagerId = h.Id  
)  
SELECT *  
FROM EmployeeHierarchy;
```

38. CTE vs Subquery

Subquery:

SQL

```
SELECT *  
FROM  
(  
    SELECT *  
    FROM Employee  
    WHERE Salary > 50000  
) e;
```

CTE:

SQL

```
WITH HighSalary AS
(
    SELECT *
    FROM Employee
    WHERE Salary > 50000
)
SELECT *
FROM HighSalary;
```

CTEs often improve readability for complex or recursive logic.

Important:

A CTE is not automatically faster than a subquery. The optimizer determines the execution plan.

39. SQL Constraints

Important constraints:

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
DEFAULT
```

Example:

SQL

```
CREATE TABLE Employee
(
    Id INT PRIMARY KEY,

    Email VARCHAR(200)
        UNIQUE
        NOT NULL,

    Age INT
        CHECK (Age >= 18),

    IsActive BIT
```

```

        DEFAULT 1,

        DepartmentId INT,

        CONSTRAINT FK_Employee_Department
        FOREIGN KEY (DepartmentId)
        REFERENCES Department (Id)

    );

```

Constraints enforce data integrity at database level.

40. Stored Procedure vs Function

Stored Procedure	Function
Executed as a procedure	Returns a defined value/table
Can support complex procedural operations	More restricted
Can return result sets/output parameters	Must return a result
Can be used for command/workflow logic	Can often participate in queries depending on DB/function type

Procedure:

SQL

```

CREATE PROCEDURE GetEmployee
    @Id INT
AS
BEGIN
    SELECT *
    FROM Employee
    WHERE Id = @Id;
END

```

Function:

SQL

```

CREATE FUNCTION GetFullName
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50)
)
RETURNS VARCHAR(101)

```

```
AS
BEGIN
    RETURN @FirstName + ' ' + @LastName;
END
```

41. SQL Profiler

SQL Server Profiler can capture activity such as:

```
Executed SQL
Stored procedure calls
Duration
Reads/writes
Errors
Login/activity events
```

Project-style answer:

When investigating a slow API, I first determine which database call is slow using application telemetry/logging. For SQL Server workloads, tracing tools can help capture the actual query or procedure execution. I then examine the execution plan, logical reads, indexes, joins and parameter behavior. In production, I prefer lower-overhead approaches such as Extended Events or Query Store rather than leaving Profiler traces running.

That sounds much stronger than simply saying "I use Profiler."

42. SQL queries returning an ID

Insert and return generated ID:

```
SQL

INSERT INTO Employee
(
    Name,
    Email
)
OUTPUT INSERTED.Id
VALUES
(
    'John',
    'john@example.com'
);
```

SQL Server also supports:

SQL

```
SELECT SCOPE_IDENTITY();
```

`OUTPUT INSERTED.Id` is often clearer and more flexible.

Query existing ID:

SQL

```
SELECT Id
FROM Employee
WHERE Email = 'john@example.com';
```

43. Constraint/key combination questions

For example:

Can a table contain:

Primary Key + Unique constraint?

Yes.

SQL

```
CREATE TABLE Users
(
    Id INT PRIMARY KEY,
    Email VARCHAR(200) UNIQUE
);
```

Can foreign key contain duplicate values?

Yes.

Department

Id = 10

Employee

Employee1 → DepartmentId 10

Employee2 → DepartmentId 10

Can a primary key contain NULL?

No.

Can a unique constraint contain NULL?

Behavior differs by database engine, so don't generalize across SQL Server and PostgreSQL.

That is exactly the kind of follow-up an experienced candidate should mention.

44. SQL Server vs PostgreSQL

Both are relational database systems, but syntax, features and implementation details differ.

Examples:

SQL Server identity:

SQL

```
Id INT IDENTITY(1,1)
```

PostgreSQL modern identity syntax:

SQL

```
id INT GENERATED ALWAYS AS IDENTITY
```

SQL Server:

SQL

```
SELECT TOP 10 *  
FROM Employee;
```

PostgreSQL:

SQL

```
SELECT *  
FROM Employee  
LIMIT 10;
```

Current timestamp:

SQL Server:

SQL

GETDATE ()

PostgreSQL:

SQL

CURRENT_TIMESTAMP

Also mention differences around JSON support, indexing options, procedural languages, tooling, query plans, data types and case-sensitivity/identifier behavior.

E. Experienced Candidate / Project Questions

45. Explain Angular → API → database flow

This answer should sound architectural rather than theoretical.

Angular Component

Authentication

↓

Authorization

↓

Routing

↓

Controller

↓

Service Layer

↓

Repository

↓

```
EF Core
  ↓
SQL Server/PostgreSQL
  ↓
Response DTO
  ↓
JSON
  ↓
Angular Service
  ↓
Signal/Observable
  ↓
Component
  ↓
UI
```

Interview answer

In my projects, the Angular component doesn't normally call backend endpoints directly. The component communicates with an Angular service, and the service uses HttpClient. Interceptors handle common HTTP concerns such as authentication headers and error handling.

The request reaches ASP.NET Core and passes through middleware for exception handling, authentication, authorization and other cross-cutting concerns. Routing maps the request to the controller. The controller delegates business logic to a service. The service communicates with the repository, and the repository uses EF Core to access SQL Server or PostgreSQL.

I generally return DTOs rather than exposing EF entities directly. The response is serialized as JSON and returned to Angular, where Observables or Signals update the UI.

This is a strong **8–15+ years experience answer**.

46. How have you implemented DI, Middleware and Repository?

Connect the three concepts.

```
Request
  ↓
Middleware
  ↓
Controller
  ↓ DI
```

```
IService
  ↓
Service
  ↓ DI
IRepository
  ↓
Repository
  ↓
DbContext
```

Example DI:

C#

```
builder.Services.AddScoped<
    IPatientService,
    PatientService>();

builder.Services.AddScoped<
    IPatientRepository,
    PatientRepository>();
```

Middleware:

C#

```
app.UseMiddleware<ExceptionMiddleware>();

app.UseAuthentication();
app.UseAuthorization();
```

Controller:

C#

```
public class PatientsController : ControllerBase
{
    private readonly IPatientService _service;

    public PatientsController(
        IPatientService service)
    {
        _service = service;
    }
}
```

Answer:

DI handles object creation and lifetime management, middleware handles cross-cutting HTTP concerns, and repositories isolate persistence/data-access concerns. Together

they keep the system loosely coupled and easier to test and maintain.

47. Investigating API failure Angular → DB

This is one of the most important scenario answers.

Do **not** immediately start debugging random backend code.

Follow the transaction systematically.

```
Angular UI
  ↓
Browser Console
  ↓
Network
  ↓
Request URL
  ↓
HTTP Method
  ↓
Headers/JWT
  ↓
Payload
  ↓
HTTP Status
  ↓
API Logs
  ↓
Middleware
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Generated SQL
  ↓
Database
```

Suppose:

```
http
POST /api/patients
```

returns:

I would:

1. Inspect Network tab.
2. Verify request URL/method.
3. Verify request payload.
4. Verify authorization header.
5. Inspect response body/correlation ID.
6. Reproduce through Swagger/API client.
7. Check centralized application logs.
8. Trace controller → service → repository.
9. Inspect generated SQL/query.
10. Run relevant query/procedure against the DB where appropriate.
11. Examine execution plan/blocking if performance-related.
12. Check external dependencies.
13. Fix and add a test/regression guard.

For distributed systems, correlation IDs are extremely useful:

```
Angular request
CorrelationId = abc123
    ↓
API
abc123
    ↓
Service
abc123
    ↓
Logs
abc123
```

You can search all related logs using the same correlation ID.

48. Which AI tools do you use in development?

For this question, avoid answering only:

"I use ChatGPT and Copilot."

Explain **where AI fits into your engineering workflow and where it doesn't replace engineering judgment.**

A strong answer:

I use AI-assisted development tools primarily for productivity rather than allowing them to make architectural decisions automatically. For example, I use GitHub Copilot for code completion, repetitive implementations, unit-test generation and refactoring suggestions. I use ChatGPT for technical analysis, troubleshooting, design discussions, SQL/query optimization ideas, documentation and exploring alternative implementations.

For AI-enabled applications, I have also worked with concepts such as Azure OpenAI, embeddings, vector search and RAG. For example, documents can be chunked, converted into embeddings, indexed in Azure AI Search and retrieved as grounding context before generating an answer.

But AI-generated code still goes through the same engineering process: code review, security validation, unit/integration tests, static analysis and performance review. I don't treat AI output as production-ready automatically.

If they ask:

"Where exactly does AI help you?"

You can answer:

```
Requirement understanding
      ↓
Architecture brainstorming
      ↓
Boilerplate generation
      ↓
Unit test generation
      ↓
Code explanation/refactoring
      ↓
SQL/query analysis
      ↓
Debugging assistance
      ↓
Documentation
      ↓
PR/code review assistance
```

For a senior/lead candidate, add:

I avoid sending credentials, secrets, production customer data or confidential information into unapproved AI tools. AI usage should follow organizational security and governance policies.

High-priority questions to prepare first

If your Infosys interview is close, I would prioritize these because they generate many follow-up questions:

1. **Web API complete request lifecycle**
2. **Middleware + middleware ordering**
3. **DI + Singleton/Scoped/Transient**
4. **Repository + Service pattern**
5. **IEnumerable vs IQueryable**
6. **Angular data binding**
7. **Signals vs RxJS/BehaviorSubject**
8. **Angular API troubleshooting**
9. **Angular → API → DB architecture**
10. **Indexes + execution plans**
11. **CTE vs subquery**
12. **Stored procedure vs function**
13. **HTTP status codes**
14. **JWT authentication/interceptors**
15. **End-to-end production debugging**

One particularly likely follow-up chain is:

```
Explain DI
↓
What are DI lifetimes?
↓
Why is DbContext Scoped?
↓
Can Singleton inject Scoped?
↓
What happens if it does?
↓
How would you resolve Scoped service
inside BackgroundService?
↓
What is IServiceScopeFactory?
```

Similarly, Angular can become:

```
What is Observable?
↓
Subject vs BehaviorSubject?
↓
What are Signals?
```

↓
Signal vs Observable?
↓
What is computed()?
↓
What is effect()?
↓
How does Angular update UI?

And SQL often becomes:

What is an Index?
↓
Clustered vs Nonclustered?
↓
Index Seek vs Scan?
↓
Composite Index?
↓
Included columns?
↓
Why isn't SQL using my index?
↓
How do you identify a slow query?
↓
Execution Plan?
↓
Parameter sniffing?

Preparing these **question chains**, rather than memorizing isolated definitions, is the better strategy for an experienced-candidate Infosys interview.